nvfp4_dual_gemm

Ended 2026-01-20 07:59 UTC

Submit via CLI

▼ Description

You will implement a block scaled dual matrix-matrix multiplication kernel with silu activation optimized for NVIDIA B200. To be explicit, you will be given a tuple of tensors:

```
(a, b1, b2, sfa, sfb1, sfb2, c)
```

where:

- `a` is M x K x L in K-major order in nvfp4(e2m1)
- `b1` is N x K x L in K-major order in nvfp4(e2m1)
- `b2` is N x K x L in K-major order in nvfp4(e2m1)
- `sfa` is M x (K // 16) x L in K-major order in fp8(e4m3fnuz)
- `sfb1` is N x (K // 16) x L in K-major order in fp8(e4m3fnuz)
- `sfb2` is N x (K // 16) x L in K-major order in fp8(e4m3fnuz)
- `c` is M x N x L in fp16

Matrix sizes `M` is divisible by mma_tiler_mn[0], `N` is divisible by mma_tiler_mn[1], `K` is divisible by 256. The ranking criteria is the geometric mean of the benchmark results. For the grand price, your kernel will be evaluated against the speed of light analysis and the solution closest to the speed of light will be awarded the grand price.

```
The speed of light analysis based on the max(FP4 Tensor Core math throughpu
  M   N   K   L time[us]
256 4096 7168 1 4.708
512 4096 7168 1 8.714
256 3072 4096 1 2.125
512 3072 7168 1 6.535
```

▼ Benchmark Shapes

- `{"k":7168,"l":1,"m":256,"n":4096}`
- `{"k":7168,"l":1,"m":512,"n":4096}`
- `{"k":4096,"l":1,"m":256,"n":3072}`

- {"k":7168,"l":1,"m":512,"n":3072}

RANKINGS REFERENCE SUBMISSION

Reference Implementation

```
import torch
from task import input_t, output_t
from utils import make_match_reference

# Scaling factor vector size
sf_vec_size = 16

# Helper function for ceiling division
def ceil_div(a, b):
    return (a + b - 1) // b

# Helper function to convert scale factor tensor to blocked format
def to_blocked(input_matrix):
    rows, cols = input_matrix.shape

    # Please ensure rows and cols are multiples of 128 and 4 respectively
    n_row_blocks = ceil_div(rows, 128)
    n_col_blocks = ceil_div(cols, 4)

    padded = input_matrix
    blocks = padded.view(n_row_blocks, 128, n_col_blocks, 4).permute(0, 2,
    rearranged = blocks.reshape(-1, 4, 32, 4).transpose(1, 2).reshape(-1, 3

    return rearranged.flatten()


def ref_kernel(
    data: input_t,
) -> output_t:
    """
    PyTorch reference implementation of NVFP4 block-scaled dual GEMM with s
    C = silu(A @ B1) * (A @ B2).
    """
    a_ref, b1_ref, b2_ref, sfa_ref_cpu, sfb1_ref_cpu, sfb2_ref_cpu, _, _, _

    # Get dimensions from MxNxL layout
```

```
    m, n, l = c_ref.shape

    # Call torch._scaled_mm to compute the GEMV result
    ref1 = torch.empty(
        (l, m, n),
        dtype=torch.float32,
        device="cuda",
    ).permute(1, 2, 0)
    ref2 = torch.empty(
        (l, m, n),
        dtype=torch.float32,
        device="cuda",
    ).permute(1, 2, 0)
    for l_idx in range(l):
        # Convert the scale factor tensor to blocked format
        scale_a = to_blocked(sfa_ref_cpu[:, :, l_idx])
        scale_b1 = to_blocked(sfb1_ref_cpu[:, :, l_idx])
        scale_b2 = to_blocked(sfb2_ref_cpu[:, :, l_idx])
        # (m, k) @ (n, k).T -> (m, n)
        res1 = torch._scaled_mm(
            a_ref[:, :, l_idx],
            b1_ref[:, :, l_idx].transpose(0, 1),
            scale_a.cuda(),
            scale_b1.cuda(),
            bias=None,
            out_dtype=torch.float32,
        )
        ref1[:, :, l_idx] = res1

        res2 = torch._scaled_mm(
            a_ref[:, :, l_idx],
            b2_ref[:, :, l_idx].transpose(0, 1),
            scale_a.cuda(),
            scale_b2.cuda(),
            bias=None,
            out_dtype=torch.float32,
        )
        ref2[:, :, l_idx] = res2
    # Do silu on the first GEMM result and multiply with the second GEMM re
    c_ref = (torch.nn.functional.silu(ref1) * ref2).to(torch.float16)
    return c_ref


def generate_input(
    m: int,
    n: int,
    k: int,
    l: int,
```

```
    seed: int,
):
    """
    Generate input tensors for NVFP4 block-scaled dual GEMM with silu activ
    C = silu(A @ B1) * (A @ B2).

    Args:
        m: Number of rows in matrix A
        n: Number of columns in matrix B1 and B2
        k: Number of columns in A and rows of B1 and B2
        l: Batch size
        seed: Random seed for reproducibility

    Returns:
        Tuple of (a, b, scale_a, scale_b, c) where:
            a: [m, k, l] - Input matrix in torch.float4e2m1fn_x2 data type
            b1: [n, k, l] - Input matrix in torch.float4e2m1fn_x2 data type
            b2: [n, k, l] - Input matrix in torch.float4e2m1fn_x2 data type
            scale_a: [m, k, l] - Input scale factors in torch.float8e4m3fn
            scale_b1: [n, k, l] - Input scale factors in torch.float8e4m3fn
            scale_b2: [n, k, l] - Input scale factors in torch.float8e4m3fn
            scale_a_permuted: [32, 4, rest_m, 4, rest_k, l] - Input scale f
            scale_b1_permuted: [32, 4, rest_n, 4, rest_k, l] - Input scale
            scale_b2_permuted: [32, 4, rest_n, 4, rest_k, l] - Input scale
            c: [m, n, l] - Output matrix in torch.float16 data type
    """
    torch.manual_seed(seed)

    def create_fp4_tensors(l, mn, k):
        # generate uint8 tensor, then convert to float4e2m1fn_x2 data type
        # generate all bit patterns
        ref_i8 = torch.randint(255, size=(l, mn, k // 2), dtype=torch.uint8

        # for each nibble, only keep the sign bit and 2 LSBs
        # the possible values are [-1.5, -1, -0.5, 0, +0.5, +1, +1.5]
        ref_i8 = ref_i8 & 0b1011_1011

        return ref_i8.permute(1, 2, 0).view(torch.float4_e2m1fn_x2)

    # Generate uint8 tensor, then convert to float4e2m1fn_x2 data type
    a_ref = create_fp4_tensors(l, m, k)
    b1_ref = create_fp4_tensors(l, n, k)
    b2_ref = create_fp4_tensors(l, n, k)
    a_ref = a_ref.view(torch.float4_e2m1fn_x2)
    b1_ref = b1_ref.view(torch.float4_e2m1fn_x2)
    b2_ref = b2_ref.view(torch.float4_e2m1fn_x2)

    # Create float16 output tensor
```

```
    c_ref = torch.randn((l, m, n), dtype=torch.float16, device="cuda").perm
        1, 2, 0
    )

    # Helper function to prepare the scale factor tensors for both referenc
    # kernel and customize kernel. The customized data layout can be found
    # https://docs.nvidia.com/cuda/cublas/index.html?highlight=fp4#d-block-
    def create_scale_factor_tensors(l, mn, sf_k):
        # Create the reference scale factor tensor (mn, sf_k, l) on CPU.
        ref_shape = (l, mn, sf_k)
        ref_permute_order = (1, 2, 0)
        # Init with fp32 tensor in [0,1), then convert to float8_e4m3fn
        ref_f8_random_fp32 = torch.rand(ref_shape, dtype=torch.float32, dev
        ref_f8_torch_tensor = ref_f8_random_fp32.to(dtype=torch.float8_e4m3
        # permute to match ref_permute_order
        ref_f8_torch_tensor_permuted = ref_f8_torch_tensor.permute(*ref_per

        atom_m = (32, 4)
        atom_k = 4
        mma_shape = (
            l,  # batch size
            ceil_div(mn, atom_m[0] * atom_m[1]),
            ceil_div(sf_k, atom_k),
            atom_m[0],
            atom_m[1],
            atom_k,
        )

        # Reorder scale factor tensor to (32, 4, rest_m, 4, rest_k, l) layo
        # Which is needed by the CuTe customized kernel
        mma_permute_order = (3, 4, 1, 5, 2, 0)
        # Generate a random int8 tensor, then convert to float8_e4m3fn
        rand_int_tensor = torch.empty(mma_shape, dtype=torch.int8, device='
        reordered_f8_torch_tensor = rand_int_tensor.to(dtype=torch.float8_e
        # Permute according to mma_permute_order
        reordered_f8_torch_tensor = reordered_f8_torch_tensor.permute(*mma_

        # GPU-side vectorized reordering (replaces slow CPU nested loops)
        # Create index grids for all dimensions
        i_idx = torch.arange(mn, device='cuda')
        j_idx = torch.arange(sf_k, device='cuda')
        b_idx = torch.arange(l, device='cuda')

        # Create meshgrid for all combinations of (i, j, b)
        i_grid, j_grid, b_grid = torch.meshgrid(i_idx, j_idx, b_idx, indexi

        # Calculate target indices in vectorized manner
        mm = i_grid // (atom_m[0] * atom_m[1])
```

```
        mm32 = i_grid % atom_m[0]
        mm4 = (i_grid % 128) // atom_m[0]
        kk = j_grid // atom_k
        kk4 = j_grid % atom_k

        # Perform the reordering with advanced indexing (all on GPU)
        reordered_f8_torch_tensor[mm32, mm4, mm, kk4, kk, b_grid] = ref_f8_

        return ref_f8_torch_tensor_permuted.cpu(), reordered_f8_torch_tenso

    sf_k = ceil_div(k, sf_vec_size)
    sfa_ref_cpu, sfa_ref_permuted = create_scale_factor_tensors(l, m, sf_k)
    sfb1_ref_cpu, sfb1_ref_permuted = create_scale_factor_tensors(l, n, sf_
    sfb2_ref_cpu, sfb2_ref_permuted = create_scale_factor_tensors(l, n, sf_

    return (a_ref, b1_ref, b2_ref, sfa_ref_cpu.to("cuda"), sfb1_ref_cpu.to(


check_implementation = make_match_reference(ref_kernel, rtol=1e-03, atol=1e
```

Discord X YouTube GitHub © 2025 GPU MODE